

Probabilistic Pairing Proofs

Anonymous Submission

Abstract. Elliptic curve pairings are fundamental to modern zero-knowledge proof systems, including Groth16, PLONK, and KZG polynomial commitments. In bidding-based execution environments (Ethereum VM, BitVM, blockchain smart contracts) where users compete for limited computational resources through gas fees, and in arithmatized computation systems (R1CS, Plonkish, Cairo AIR) where each computational step imposes significant prover overhead, the complexity of pairings computation remains a practical problem.

We present a public-coin interactive proof system that consolidates the Miller loop iterations and final exponentiation into a single polynomial relation. The relation can then be verified as polynomial identities greatly reducing the pairing costs.

The techniques presented here have been integrated into Garaga [FE], a production elliptic curve tooling library for Cairo and Starknet. For BN254 3-pairing verification (standard in Groth16), the implementation achieves: 45% reduction in field operations ($14,456 \rightarrow 7,975$ MULMOD), 77% reduction in commitment overhead ($3,758 \rightarrow 876$ POSEIDON hashes), and 37% overall speedup. These improvements enable practical pairing verification in resource-constrained blockchain environments and recursive proof systems.

Keywords: Elliptic curve pairings · Zero-knowledge proofs · Polynomial identity testing · Blockchain verification · Recursive proofs.

1 Introduction

Blockchain technology continues to mature with improving scalability and infrastructure. However, privacy remains a significant barrier to entry for many users and applications. Zero-knowledge proofs (ZKPs) have emerged as a powerful solution to this challenge.

1.1 Motivation

Elliptic curve pairings represent fundamental building blocks for numerous cryptographic systems including Groth16[Gro16], PLONK[GWC19], BLS signature schemes[BGW⁺22], and KZG polynomial commitment schemes[KZG10].

However, **pairings are computationally intensive**. This creates critical bottlenecks in two important deployment contexts:

1. **Resource-constrained virtual machines** (Ethereum VM, BitVM, ZKVMs): Computation is metered — each operation incurs gas costs. A single pairing verification requiring thousands of field operations becomes economically prohibitive during peak network congestion.
2. **Arithmatized computation systems** (R1CS [GGPR13], Plonkish [GWC19], Cairo AIR [GPR21]): Each operation must be represented algebraically, adding significant prover overhead beyond native execution costs.

In such contexts, we can make pairing application practical by using an efficient proof system to verify the correctness of the pairing computation, rather than executing all operations natively within the expensive environment.

1.2 Our Contribution

We present a public-coin interactive proof system for verifying pairing computations that consolidates verification into unified polynomial relationships under the Schwartz-Zippel lemma. By treating complete Miller loop iterations as single polynomial relationships rather than sequences of individual extension field operations [Fel23], our approach achieves:

- **45% reduction** in Miller loop field operations
- **77% reduction** in commitment overhead (Fiat-Shamir hashing)
- **37% overall speedup** for complete pairing verification

Our key insight is that instead of verifying individual field operations separately, we can verify entire Miller loop iterations as single polynomial identities. This consolidation dramatically reduces both the number of modular operations required and the communication overhead from intermediate commitments.

2 Background

Algorithm 1 Optimal Ate pairing over Barreto-Naehrig curves [BGM⁺10]

Input: $P \in \mathbb{G}_1, Q \in \mathbb{G}_2$

Output: $a_{\text{opt}}(Q, P)$

```

1: Write  $s = 6t + 2 = \sum_{i=0}^{L-1} s_i 2^i$  where  $s_i \in \{-1, 0, 1\}$  and  $s_L = 1$ 
2:  $T \leftarrow Q, f \leftarrow 1$ 
3: for  $i = L - 2$  to  $0$  do
4:    $f \leftarrow f^2 \cdot l_{T,T}(P)$ 
5:    $T \leftarrow [2]T$ 
6:   if  $s_i = 1$  then
7:      $f \leftarrow f \cdot l_{T,Q}(P)$ 
8:      $T \leftarrow T + Q$ 
9:   end if
10:  if  $s_i = -1$  then
11:     $f \leftarrow f \cdot l_{T,-Q}(P)$ 
12:     $T \leftarrow T - Q$ 
13:  end if
14: end for
15:  $Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$ 
16:  $f \leftarrow f \cdot l_{T,Q_1}(P), T \leftarrow T + Q_1$ 
17:  $f \leftarrow f \cdot l_{T,-Q_2}(P), T \leftarrow T - Q_2$ 
18:  $f \leftarrow f^{p^{12}-1/r}$ 
19: return  $f$ 

```

2.1 Pairing Computation

An elliptic curve pairing is a non-degenerate bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, where $\mathbb{G}_1$ and $\mathbb{G}_2$ are groups of points on elliptic curves over finite fields, and $\mathbb{G}_T$ is a multiplicative group in an extension field. For practical cryptographic applications, we require that this map be efficiently computable. We will focus on the optimal ate pairing [BGM⁺10] over Barreto-Naehrig (BN) curves, which is widely used due to its efficiency. We will specifically

use the Ethereum’s BN254 curve, defined over a 254-bit prime field $\mathbb{F}_q$, with groups $\mathbb{G}_1, \mathbb{G}_2$ of order r and target group $\mathbb{G}_T$ contained in the 12th extension field $\mathbb{F}_{q^{12}}$.

The computation of an optimal ate pairing—the focus of our implementation—consists of two primary phases:

1. **Miller loop:** The core iterative computation that evaluates line functions at pairs of points from $\mathbb{G}_1$ and $\mathbb{G}_2$, accumulating intermediate results to produce a value in $\mathbb{F}_{q^{12}}$.
2. **Final Exponentiation:** This step raises the Miller loop output to the power $(q^{12} - 1)/r$, where r is the order of the groups, ensuring the result lies in the correct subgroup $\mathbb{G}_T$.

While both phases present optimization opportunities, the constraint-based verification of these computations introduces unique challenges and possibilities that differ significantly from native implementations.

2.2 Schwartz-Zippel lemma

The Schwartz-Zippel lemma (or more precisely, the Demillo-Lipton-Schwartz-Zippel lemma [DL78, Sch79, Zip79, Sch80]) provides a probabilistic method for testing polynomial identities. The lemma states that for a non-zero polynomial $P(x_1, x_2, \dots, x_n)$ of total degree d over a finite field $\mathbb{F}$, when evaluated at uniformly random field elements, the probability that P evaluates to zero is at most $d/|\mathbb{F}|$.

This probabilistic bound becomes negligible for cryptographically sized fields where $d \ll |\mathbb{F}|$, making it practical to verify polynomial identities by evaluation instead of coefficient-wise operations.

2.3 Computation Models

While we specifically implement our library in Cairo, we map our MULMOD operation to an constraint (denoted by $\mathbf{C}$) to maintain compatibility with the Rank-1 Constraint System (R1CS) model to benchmark against established literature. In R1CS [Gro16], prover complexity is dominated by multiplication gates while additions are free. In the Cairo VM, this corresponds to the Algebraic Intermediate Representation (AIR) where operations are verified through polynomial constraints. To synchronize our data with the benchmarks provided by El Housni [Hou23], we define our cost metric based on field multiplications.

2.3.1 R1CS Cost Model

We measure computational cost in terms of *constraints* (denoted by $\mathbf{C}$), representing the fundamental unit of work in Rank-1 constraint systems. One constraint corresponds to one multiplication gate.

Basic Field Operations in $\mathbb{F}_q$:

- **Addition/Subtraction:** Free
- **Multiplication:** One constraint (1C)
- **Division/Inversion:** One multiplication + one equality check

Commitment Overhead: Beyond constraint costs, we account for *calldata* or *commitment overhead*—the number of field elements the prover must commit to (via Fiat-Shamir hashing in non-interactive proofs). For blockchain deployments, this overhead directly impacts transaction costs and is often the dominant factor.

2.3.2 Cost Model Comparison

Table 1 contrasts costs between native implementations and constraint systems:

Table 1: Cost comparison: Native vs Constraint-based computation

Operation	Native Cost	Constraint Cost	Ratio
$\mathbb{F}_q$ Multiplication	$1\times$	1C	$1\times$
$\mathbb{F}_q$ Inversion	$100\times$	2C	$\approx 50\times$ cheaper
$\mathbb{F}_{q^{12}}$ Multiplication	$50\times$	54C (traditional)	Similar
$\mathbb{F}_{q^{12}}$ Multiplication	$50\times$	39C (our method)	Better

Key Insight: Operations that are traditionally expensive (inversions, divisions) become cheap in constraint systems when computed via witnesses. This inverts traditional optimization priorities: we can afford inversions to save multiplications, enabling affine coordinate systems and other techniques impractical in native implementations.

Recognizing and exploiting these unique computational properties is essential for unlocking hidden performance potential. Youssef El Housni’s pioneering work **Pairings in Rank-1 Constraint Systems** [Hou23] represents, to our knowledge, the first systematic exploration of these optimization opportunities, providing a breakthrough in the practicality of recursive proving by thoughtfully leveraging the distinctive cost model inherent to constraint-based systems.

2.4 Related Work

The landscape of efficient pairing verification in constraint systems has evolved through several key contributions. El Housni’s **Pairings in Rank-1 Constraint Systems** [Hou23] established the theoretical framework for optimizing pairings in Rank-1 Constraint Systems, introducing techniques for efficient Miller loop implementation and final exponentiation that exploit the unique properties of constraint-based computation models. This work demonstrated that operations traditionally considered expensive in native implementations could be restructured to achieve remarkable efficiency gains in arithmetized environments.

Feltroidprime’s subsequent research on **enhanced extension field multiplication techniques** [Fel23] introduced polynomial verification methods using the Schwartz-Zippel lemma for $\mathbb{F}_{q^{12}}$ arithmetic operations. By representing extension field elements as polynomials and verifying their operations through polynomial arithmetic, this approach achieved significant performance improvements in the pairing verification.

Novakovic and Eagen’s work **On Proving Pairings** [NE24] presented breakthrough optimization strategies that enable the elimination of final exponentiation overhead through residue witness techniques. Their approach demonstrates that two elements in $\mathbb{F}_{q^{12}}$ can be proven equivalent without expensive final exponentiation by introducing witness elements that satisfy specific algebraic relationships, and effectively embedding the final exponentiation verification into the Miller loop computation itself.

3 Pairing Operations as Polynomials

The techniques discussed in the previous section are outlined in Algorithm 2. This algorithm combines El Housni’s R1CS optimizations [Hou23] with Novakovic and Eagen’s residue witness technique for final exponentiation elimination [NE24]. The algorithm uses a **27th root of unity** W to scale the Miller loop output $f \in \mathbb{F}_{q^{12}}$. f needs to be a cubic residue for final exponentiation (see Section 4.3 [NE24]). If f is not a cubic residue, multiplying it by W^w , $w \in 1, 2$ makes it one.

142 We now turn our attention to our primary contribution. Building upon Feltroidprime's
 143 direct extension multiplication framework, we present a new optimization that significantly
 144 improves efficiency across all phases of the pairing computation. Our approach consolidates
 145 operations across the entire process into unified polynomial verifications. This includes
 146 the Miller loop step operations as well as the final exponentiation (via residue witness)
 147 verification. Each phase is expressed as a polynomial relationship and verified using
 148 the Schwartz-Zippel lemma, substantially reducing constraint count while maintaining
 149 cryptographic security.

Algorithm 2 Optimal Ate pairing with Residue witness check

Input: $P \in \mathbb{G}_1, Q \in \mathbb{G}_2, c \in \mathbb{F}_{q^{12}}, w \in \{0, 1, 2\}$
 Internal $W, W^2 \in \mathbb{F}_{q^{12}}$ ▷ W is 27th root of unity
Output: $a_{opt}(Q, P)$

- 1: Write $s = 6t + 2 = \sum_{i=0}^{L-1} s_i 2^i$ where $s_i \in \{-1, 0, 1\}$ and $s_L = 1$
- 2: $c^{-1} \leftarrow 1/c$ ▷ Inverse Residue witness
- 3: $f \leftarrow c^{-1}$ ▷ Initialize with inverse residue witness
- 4: $T \leftarrow Q$ ▷ T_i for each pair for multi-pairing
 Every operation involving T, P or Q is repeated for T_i, P_i and Q_i for multi-pairing.
- 5: **for** $i = L - 2$ to 0 **do**
- 6: $f \leftarrow f^2$
- 7: $f \leftarrow f \cdot l_{T,T}(P)$ ▷ Repeat for T_i, P_i
- 8: $T \leftarrow [2]T$ ▷ Repeat for T_i
- 9: **if** $s_i = 1$ **then**
- 10: $f \leftarrow f \cdot c^{-1}$ ▷ Residue witness
- 11: $f \leftarrow f \cdot l_{T,Q}(P)$ ▷ Repeat for T_i, Q_i, P_i
- 12: $T \leftarrow T + Q$ ▷ Repeat for T_i, Q_i
- 13: **else if** $s_i = -1$ **then**
- 14: $f \leftarrow f \cdot c$ ▷ Residue witness
- 15: $f \leftarrow f \cdot l_{T,-Q}(P)$ ▷ Repeat for $T_i, -Q_i, P_i$
- 16: $T \leftarrow T - Q$ ▷ Repeat for $T_i, -Q_i$
- 17: **end if**
- 18: **end for**
- 19: $Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$ ▷ Repeat for Q_{1i}, T_i
- 20: $f \leftarrow f \cdot l_{T,Q_1}(P)$ ▷ Repeat for Q_{1i}, T_i, P_i
- 21: $T \leftarrow T + Q_1$ ▷ Repeat for Q_{1i}, T_i
- 22: $f \leftarrow f \cdot l_{T,-Q_2(Q)}(P)$ ▷ Repeat for Q_i, T_i, P_i
▷ Skip $T \leftarrow T - Q_2(Q)$
- 23: **if** $w = 1$ **then**
- 24: $f \leftarrow f \cdot W$
- 25: **else if** $w = 2$ **then**
- 26: $f \leftarrow f \cdot W^2$
- 27: **end if**
- 28: $f \leftarrow f \cdot c^{-q} \cdot c^{q^2} \cdot c^{-q^3}$ ▷ Uses Frobenius maps, Negative exponents use c^{-1}
- 29: **return** f

3.1 Unified Polynomial Framework

Any sequence of field operations in $\mathbb{F}_{q^{12}}$ ¹ can be expressed as a single polynomial equation of the form:

$$A_1(x) \cdot \dots \cdot A_n(x) = Q(x) \cdot P_{12}(x) + R(x) \text{ where } A_1, \dots, A_n, R \in \mathbb{F}_{q^{12}}$$

This unified framework allows us to batch verify entire computational phases rather than individual operations, achieving dramatic improvements in both constraint counts and commitment overhead across the complete pairing computation.

3.2 Comparative Analysis

The Miller loop is comprised of steps for each individual bit, which in NAF (non-adjacent form)[DHM04], may be +1, -1 or 0. Instead of performing a polynomial commitment for each individual extension field arithmetic operation separately, we propose the whole bit operation be viewed as one polynomial. And commitment be made to that polynomial.

To get a closer look into effectiveness of our scheme, we will make a real world application comparison for verification of three—pairing Miller loop which is common for verification of Groth16[Gro16].

For the comparison equations below, $F \in \mathbb{F}_{q^{12}}$ represents the Miller loop accumulator at the start of each bit operation (line 6). $L_* \in \text{Sparse}_{01379}$ are the sparse line function outputs—either doublings $l_{T_i, T_i}(P_i)$ or as clarified in the section—for the three pairs. $R(x) \in \mathbb{F}_{q^{12}}$ is the remainder polynomial (the updated accumulator), and $Q(x)$ is the quotient polynomial.

3.2.1 Previous scheme

Adapting faster extension field multiplication [Fel23] idea to line 6 and 7 of Miller loop algorithm 2 for a 3-pair Miller loop, we have following polynomial relationships to assert.

$$F(x) \cdot F(x) \stackrel{?}{=} Q_1(x) \cdot P_{12}(x) + T_1(x)$$

$$T_1(x) \cdot L_1(x) \stackrel{?}{=} Q_2(x) \cdot P_{12}(x) + T_2(x)$$

$$T_2(x) \cdot L_2(x) \stackrel{?}{=} Q_3(x) \cdot P_{12}(x) + T_3(x)$$

$$T_3(x) \cdot L_3(x) \stackrel{?}{=} Q(x) \cdot P_{12}(x) + R(x)$$

F, T_1, T_2, T_3, R cost 12C each for evaluation, 4C for $\mathbb{F}_q$ multiplication, and L_1, L_2, L_3 require 4C each for evaluation. All the remainder hints need to be provided for commitment, so this is 12 calldata commitment for R and intermediate remainder polynomials T_1, T_2, T_3 . **Costs sum up to 76C** ($5 * 12C + 3 * 4C + 4C$) **and 48 commitments** in $\mathbb{F}_q$ ($4 * 12$). Resulting remainder polynomial $R \in \mathbb{F}_{q^{12}}$ is the result of the Miller loop step.

3.2.2 Our scheme

In our approach, instead of working with individual arithmetic operations, we treat the entire bit operation as a single polynomial. In our scheme line 6 and 7 of 3-pair Miller loop (Algorithm 2) looks like this.

$$F(x) \cdot F(x) \cdot L_1(x) \cdot L_2(x) \cdot L_3(x) \stackrel{?}{=} Q(x) \cdot P_{12}(x) + R(x)$$

¹Elements need not have all 12 non-zero coefficients, but should align with the $\mathbb{F}_{q^{12}}$ representation (e.g., sparse line functions).

Similar to the previous approach, F and R cost $12C$ each for evaluation, $4C$ for $\mathbb{F}_q$ multiplication, and L_1, L_2, L_3 require $4C$ each for evaluation, but we eliminate intermediate (T_* polynomials above) commitments and evaluations. Only the remainder hint needs commitment, so this is 12 calldata commitment for R . **Costs sum up to $40C$ ($2 * 12C + 3 * 4C + 4C$) and 12 $\mathbb{F}_q$ commitments.**

This results in **47% fewer constraints** and a whopping **75% savings in the commitments**. When implemented on a blockchains for verification of ZK proofs, commitment savings become far more critical because of limitations and costs associated with calldata needed for hints.

3.2.3 Performance Comparison

Table 2 compares the computational costs and commitment overhead between the schemes for a single 0-bit Miller loop operation with 3 pairs.

Table 2: 3-Pair Miller loop 0-bit constraints comparison

Scheme	Constraints	$\mathbb{F}_q$ Commitments	Improvement
[Hou23]	132C	—	Baseline
[Fel23]	76C*	48	42%
Our scheme	40C*	12	70%
[Hou23]: 1 square ($36C$) + 3 sparse mul ($3 \times 30C$) = $132C$ [Fel23]: 5 evals ($5 \times 12C$) + 3 line evals ($3 \times 4C$) + 4 muls ($4 \times 1C$) = $76C$ Our scheme: 2 evals ($2 \times 12C$) + 3 line evals ($3 \times 4C$) + 4 muls ($4 \times 1C$) = $40C$ *Costs exclude Fiat-Shamir commitment overhead			

3.3 Operation Polynomials

Our implementation uses different polynomials for different bits in the Miller loop. Let us walk through a 3-pairing setup which Inversions used widely for Groth16 verification. We will describe the polynomials for zero bits, non-zero bits and correction step.

3.3.1 Miller loop: Zero Bit Equation

As seen in 3.2.2 for zero bit operations (line 6 and 7 with three $l_{T,T}(P)$ lines in Algorithm 2), the verification equation is:

$$F^2(x) \cdot L_1(x) \cdot L_2(x) \cdot L_3(x) = R(x) + Q(x) \cdot P_{12}(x) \quad (1)$$

Table 3: Zero bit operation polynomial components

Polynomial	Description	Degree	Coefficients
$F \in \mathbb{F}_{q^{12}}$	Miller loop accumulator	$11 \times 2 = 22$	12
$L_1, L_2, L_3 \in \text{Sparse}_{01379}$	The line functions	$9 \times 3 = 27$	$4 \times 3 = 12$
$R \in \mathbb{F}_{q^{12}}$	Remainder, The result	11	12
Total constraints	36C Evals, 4C Muls and 1C RLC		41
Q (Amortized See 4.1)	$\deg(\text{LHS}) - \deg(P_{12}) + 1$	38	39

3.3.2 Miller loop: Non-Zero Bit Equation

For non-zero bit operations (lines 10, 11, 14, 15 with three pairs of lines in Algorithm 2), the Miller loop performs both doubling and addition steps. So for each pair we have two Sparse_{01379} lines. The verification equation becomes:

$$F^2(x) \cdot W(x) \cdot L_{d1}(x) \cdot L_{d2}(x) \cdot L_{d3}(x) \cdot L_{a1}(x) \cdot L_{a2}(x) \cdot L_{a3}(x) = R(x) + Q(x) \cdot P_{12}(x) \quad (2)$$

Table 4: Non-zero bit operation polynomial components

Polynomial	Description	Degree	Coefficients
$F \in \mathbb{F}_{q^{12}}$	Miller loop accumulator	$11 \times 2 = 22$	12
$L_{d1}, L_{d2}, L_{d3} \in \text{Sparse}_{01379}$	Doubling line functions	$9 \times 3 = 27$	$4 \times 3 = 12$
$L_{a1}, L_{a2}, L_{a3} \in \text{Sparse}_{01379}$	Addition line functions	$9 \times 3 = 27$	$4 \times 3 = 12$
W or $W^{-1} \in \mathbb{F}_{q^{12}}$	Residue witness	11	12
$R \in \mathbb{F}_{q^{12}}$	Remainder, The result	11	12
Total constraints	60C Evals, 8C Muls and 1C RLC		69
Q (<i>Amortized See 4.1</i>)	$\deg(\text{LHS}) - \deg(P_{12}) + 1$	76	77

The residue witness W encapsulates the equivalence class relationship established by Novakovic and Eagen’s final exponentiation elimination. The inverse of W is used for negative bits, this just changes the coefficients the equation remains the same.

3.3.3 Miller loop: Frobenius mapping Equation

The correction step finalizes the Miller loop by incorporating the Frobenius endomorphism corrections (lines 20 and 22 in Algorithm 2). Again, we have two Sparse_{01379} lines for each pair, for π_p and $-\pi_p^2$ mappings, without any squaring of the accumulator. The verification equation becomes:

$$F(x) \cdot L_{1\pi}(x) \cdot L_{2\pi}(x) \cdot L_{3\pi}(x) \cdot L_{1\pi^2}(x) \cdot L_{2\pi^2}(x) \cdot L_{3\pi^2}(x) = R(x) + Q(x) \cdot P_{12}(x) \quad (3)$$

Table 5: Correction step polynomial components

Polynomial	Description	Degree	Coefficients
$F \in \mathbb{F}_{q^{12}}$	Miller loop accumulator	11	12
$L_{1\pi}, L_{2\pi}, L_{3\pi} \in \text{Sparse}_{01379}$	π_p correction lines	$9 \times 3 = 27$	$4 \times 3 = 12$
$L_{1\pi^2}, L_{2\pi^2}, L_{3\pi^2} \in \text{Sparse}_{01379}$	$-\pi_p^2$ correction lines	$9 \times 3 = 27$	$4 \times 3 = 12$
$R \in \mathbb{F}_{q^{12}}$	Remainder, The result	11	12
Total constraints	48C Evals, 6C Muls and 1C RLC		55
Q (<i>Amortized See 4.1</i>)	$\deg(\text{LHS}) - \deg(P_{12}) + 1$	54	55

3.3.4 Final Exponentiation Equation

The final exponentiation check is handled as described in [NE24]. The equation handles the cubic scale factor multiplication, and **Frobenius component** ($q - q^2 + q^3$) of the final exponentiation (lines 24, 26 and 28 in Algorithm 2). The verification equation looks like:

$$F(x) \cdot c^{-q}(x) \cdot c^{q^2}(x) \cdot c^{-q^3}(x) \cdot W^w(x) = 1 + Q(x) \cdot P_{12}(x) \quad (4)$$

where $w \in \{0, 1, 2\}$ determines the cubic root of unity scaling factor, and the Frobenius mappings are computed using the residue witness c and its inverse and provided as input. Cubic root is sparse element with just 2 coefficients the degree ranging from 8 to 10 because of positioning of the coefficients. Our R is now 1 for a successful proof verification.

Table 6: Final exponentiation polynomial components

Polynomial	Description	Degree	Coefficients
$F \in \mathbb{F}_{q^{12}}$	Final Miller loop result	11	12
$c^{-q} \in \mathbb{F}_{q^{12}}$	Residue witness Frobenius $-q$	11	12
$c^{q^2} \in \mathbb{F}_{q^{12}}$	Residue witness Frobenius q^2	11	12
$c^{-q^3} \in \mathbb{F}_{q^{12}}$	Residue witness Frobenius $-q^3$	11	12
$W^w \in \mathbb{F}_{q^{12}}$	Cubic scale factor	10	2
$R = 1$	Unity (constant)	0	1
Total constraints	51C Evals, 4C Muls and 1C RLC		56
Q (<i>Amortized See 4.1</i>)	$\deg(\text{LHS}) - \deg(P_{12}) + 1$	43	44

With this we have all the polynomials we need for verifying entire pairing operation. Next we will look into high level overview of pairing verification via an interactive oracle proof.

3.4 Security

We analyze the security of our polynomial verification scheme.

3.4.1 Polynomial Identity Testing

A polynomial P of degree d over a field $\mathbb{F}_q$ has at most d roots in $\mathbb{F}_q$. Thus for a non-zero univariate polynomial evaluated at a random point $r \xleftarrow{\$} \mathbb{F}_q$, the probability of zero output is,

$$\Pr[P(r) = 0] \leq \frac{d}{|\mathbb{F}_q|}$$

3.4.2 Euclidean Division

Given polynomials A and B in $\mathbb{F}_q[x]$ with $B \neq 0$, there exist unique polynomials Q and R where $\deg(R) < \deg(B)$ such that:

$$A(x) = B(x) \cdot Q(x) + R(x)$$

3.4.3 Security Analysis

Our polynomials are of the form $P = A_1(x) \cdot A_2(x) \cdots A_n(x) - Q(x) \cdot P_{12}(x) - R(x)$ where $\deg(R) \leq 11$ and $\deg(P) < 2^8$. For verifying the computation of R with respect to P_{12} :

Soundness: The soundness error is $\delta_s = \Pr[P(r) = 0] \leq \frac{d}{|\mathbb{F}_q|}$ where $d = \deg(P) < 2^{8^2}$. For the 254-bit BN254 field, $|\mathbb{F}_{q^r}| \approx 2^{254}$, so $\delta_s \leq 2^{-246}$.

Completeness: If the prover is honest and provides correct Q and R , then $P(x) = 0$ for all $x \in \mathbb{F}_q$. Thus, the verifier always accepts.

²For k -pair pairings, the maximum degree is approximately $88 + 18(k - 3)$, remaining well below 2^8 for most practical purposes. But can be increased to 2^{10} for many more pairs with negligible change in soundness.

4 Full Pairing Proof with Quotients batching

Now that we have covered the core construction behind our contribution, let's take a holistic view of verification of a pairing computation with an interactive proof system. While the construction above works beautifully on its own, we can further optimize the protocol by batching all polynomial verifications into a two round interactive proof.

4.1 Batching Polynomial Identity Testing

The whole process of polynomial verification can be batched into a single polynomial verification.

The prover can commit all R s for all n equations involved in pairing computation in the first interaction and receive a random challenge z . The challenge z can then be used to batch all the Q s into a single polynomial as a consecutive powers random linear combination like this,

$$Q_{\text{acc}} = \sum_{i=0}^{n-1} z^i \cdot Q_i(z)$$

The verifier then prepares a new challenge c and checks,

$$\sum_{i=0}^{n-1} z^i \cdot A_i(c) \cdot B_i(c) \cdots X_i(c) = \sum_{i=0}^{n-1} z^i \cdot R_i(c) + Q_{\text{acc}}(c) \cdot P_{12}(c)$$

or,

$$\sum_{i=0}^{n-1} z^i \cdot [A_i(c) \cdot B_i(c) \cdots X_i(c) - R_i(c)] = Q_{\text{acc}}(c) \cdot P_{12}(c) \quad (5)$$

This reduces n polynomial evaluations of degree ranging from 38 to 76 (as described in Tables 3, 4, 5, 6) to a single polynomial of degree 76.

4.2 Prover's Algorithms

The prover computes quotients Q_i and remainders R_i for each polynomial equation described in Section 3.3. Given input point pairs $P \in \mathbb{G}_1$ and $Q \in \mathbb{G}_2$, the residue witness c as computed in [NE24], and internal parameter W (the 27th root of unity), the prover runs the pairing computation while simultaneously performing Euclidean division (Algorithm 3) at each step to produce the verification polynomials. This process is detailed in Algorithm 5. After receiving the challenge z from the verifier, the prover computes the random linear combination (Algorithm 4) of all quotients Q_i to produce Q_{acc} . Prover then sends Q_{acc} and all remainders R_i to the verifier.

4.3 Verifier's Algorithms

The verifier receives point pairs $P \in \mathbb{G}_1$ and $Q \in \mathbb{G}_2$, the residue witness c from [NE24], the internal parameter W (the 27th root of unity), computed remainders $\mathcal{R}$ for each polynomial equation described in Section 3.3 and Q_{acc} from the prover. The verifier retains the challenge c from the first round. Using these values, the verifier performs a pairing computation with polynomial evaluations (Algorithm 6) at each step, as detailed in Algorithm 7. After each evaluation, the verifier attempts to reconstruct RLC Q_{acc} the same way as the prover did, And checks the final equality described in Equation 5 on page 10.

Algorithm 3 PolyMulDiv: Polynomial multiplication and Euclidean division

Input: $P_1, P_2, \dots, P_n \in \mathbb{F}_q[x]$
 Internal: Divisor $P_{12} \in \mathbb{F}_q[x]$
Output: $Q, R \in \mathbb{F}_q[x]$ where $\deg(R) < \deg(P_{12})$ and $\prod_{i=1}^n P_i = Q \cdot P_{12} + R$

- 1: $Q \leftarrow 0, R \leftarrow P_1$ ▷ Initialize remainder with first polynomial, Q as 0
- 2: **for** $i = 2$ to n **do** ▷ Start from second polynomial
- 3: $R \leftarrow R \cdot P_i$ ▷ Multiply polynomials into R
- 4: **end for**
- 5: $d_P \leftarrow \deg(P_{12}), d_R \leftarrow \deg(R), \ell_P \leftarrow \text{leading_coeff}(P_{12})$
- 6: **while** $d_R \geq d_P$ **do** ▷ Euclidean division loop
- 7: $t \leftarrow \text{leading_coeff}(R) / \ell_P \cdot x^{d_R - d_P}$
- 8: $Q \leftarrow Q + t, R \leftarrow R - t \cdot P_{12}, d_R \leftarrow \deg(R)$
- 9: **end while**
- 10: **return** (Q, R) ▷ Quotient and Remainder with $\deg(R) < \deg(P_{12})$

Algorithm 4 Random Linear Combination: Adjacent Powers

Input: $P_1, P_2, \dots, P_n \in \mathbb{F}_q[x], z \in \mathbb{F}_q$
Output: $\sum_{i=1}^n z^{i-1} P_i(z)$

- 1: $P_{\text{acc}} \leftarrow P_1$ ▷ Initialize P_{acc} with first polynomial
- 2: **for** $i = 2$ to n **do** ▷ Start from second polynomial
- 3: $P_{\text{acc}} \leftarrow P_{\text{acc}} + z^{i-1} P_i(z)$
- 4: **end for**
- 5: **return** P_{acc}

284 **4.4 Interactive proof**

285 Let's look into how the algorithms describe above come together to prove and verify a
 286 pairing computation. We will go through the two-round public-coin interactive proof system
 287 in this section and in the next section we will see how to convert it into a non-interactive
 288 proof using Fiat-Shamir transformation.

Algorithm 5 Pairing Prover: Preparing polynomials

Input: $P \in \mathbb{G}_1, Q \in \mathbb{G}_2, c \in \mathbb{F}_{q^{12}}, w \in \{0, 1, 2\}$
 Internal: $W, W^2 \in \mathbb{F}_{q^{12}}$ $\triangleright W$ is 27th root of unity

Output: Arrays $\mathcal{Q}, \mathcal{R}$ of quotients and remainders
 Write $s = 6t + 2 = \sum_{i=0}^{L-1} s_i 2^i$ where $s_i \in \{-1, 0, 1\}$ and $s_L = 1$

- 1: $c^{-1} \leftarrow 1/c$ $\triangleright$ Inverse Residue witness
- 2: $f \leftarrow c^{-1}$ $\triangleright$ Initialize with inverse residue witness
- 3: $T \leftarrow Q$ $\triangleright T_i$ for each pair for multi-pairing

Every operation involving T, P or Q is repeated for T_i, P_i and Q_i for multi-pairing.

Miller loop bits

- 4: **for** $i = L - 2$ to 0 **do**
- 5: $L_{\text{dbl}} \leftarrow l_{T,T}(P), T \leftarrow [2]T$
- 6: **if** $s_i = 0$ **then**
- 7: $Q, R \leftarrow \text{PolyMulDiv}(f, f, L_{\text{dbl}})$ $\triangleright$ Algorithm 3, add L_{*i} for each P_i, Q_i
- 8: **else if** $s_i = 1$ **then**
- 9: $L_{\text{add}} \leftarrow l_{T,Q}(P), T \leftarrow T + Q$
- 10: $Q, R \leftarrow \text{PolyMulDiv}(f, f, c^{-1}, L_{\text{dbl}}, L_{\text{add}})$ $\triangleright$ Algorithm 3, add L_{*i} for each P_i, Q_i
- 11: **else if** $s_i = -1$ **then**
- 12: $L_{\text{add}} \leftarrow l_{T,-Q}(P), T \leftarrow T - Q$
- 13: $Q, R \leftarrow \text{PolyMulDiv}(f, f, c, L_{\text{dbl}}, L_{\text{add}})$ $\triangleright$ Algorithm 3, add L_{*i} for each P_i, Q_i
- 14: **end if**
- 15: $f \leftarrow R, \mathcal{Q}.\text{append}(Q), \mathcal{R}.\text{append}(R)$
- 16: **end for**

Miller loop Frobenius correction

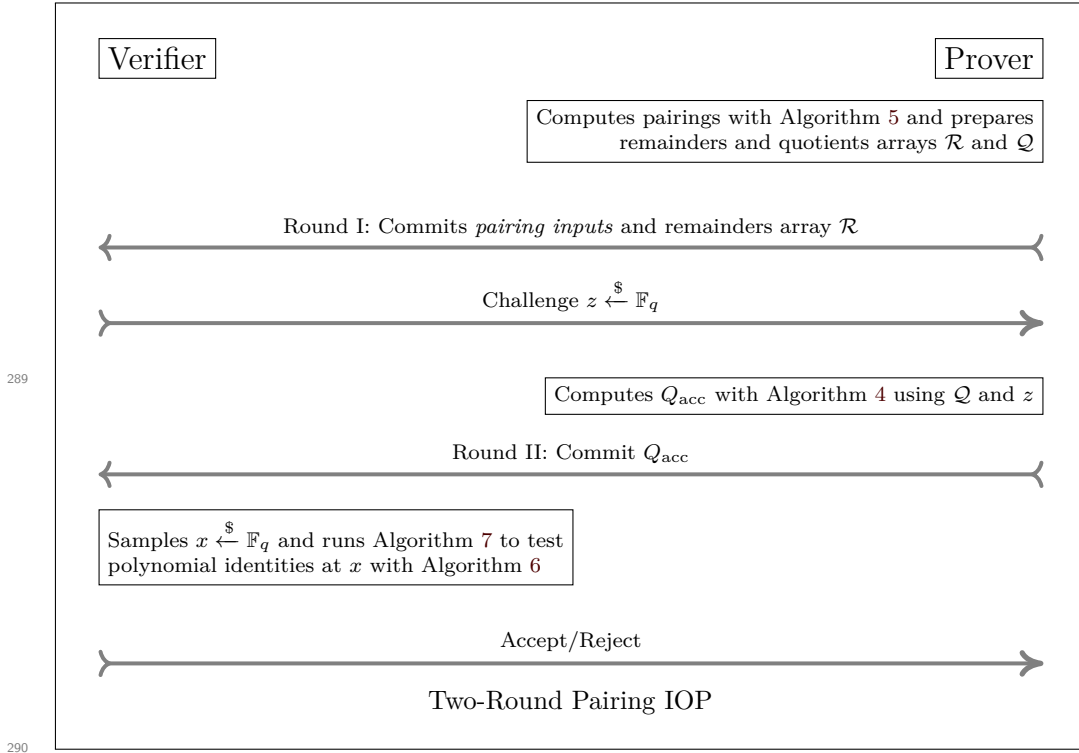
- 17: $Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$
- 18: $L_\pi \leftarrow l_{T,Q_1}(P)$
- 19: $T \leftarrow T + Q_1$
- 20: $L_{-\pi^2} \leftarrow l_{T,-Q_2}(P)$
- 21: $Q, R \leftarrow \text{PolyMulDiv}(f, L_\pi, L_{-\pi^2})$ $\triangleright$ Algorithm 3, add L_{*i} for each P_i, Q_i
- 22: $f \leftarrow R, \mathcal{Q}.\text{append}(Q), \mathcal{R}.\text{append}(R)$

Final exponentiation: Cubic scaling + Frobenius mappings

- 23: **if** $w = 0$ **then**
- 24: $Q, R \leftarrow \text{PolyMulDiv}(f, c^{-q}, c^{q^2}, c^{-q^3})$ $\triangleright$ Algorithm 3
- 25: **else if** $w = 1$ **then**
- 26: $Q, R \leftarrow \text{PolyMulDiv}(f, W, c^{-q}, c^{q^2}, c^{-q^3})$ $\triangleright$ Algorithm 3
- 27: **else if** $w = 2$ **then**
- 28: $Q, R \leftarrow \text{PolyMulDiv}(f, W^2, c^{-q}, c^{q^2}, c^{-q^3})$ $\triangleright$ Algorithm 3
- 29: **end if**
- 30: $\mathcal{Q}.\text{append}(Q), \mathcal{R}.\text{append}(R)$ $\triangleright$ Final $\mathcal{Q}$ and $\mathcal{R}$ accumulation, skip f
- 31: **return** $\mathcal{Q}, \mathcal{R}$

Algorithm 6 EvalPoly: Evaluate polynomial product minus remainder**Input:** $R \in \mathbb{F}_q[x]$, $z \in \mathbb{F}_q$, $P_1, P_2, \dots, P_n \in \mathbb{F}_q[x]$ **Output:** $e \in \mathbb{F}_q = \prod_{i=1}^n P_i(z) - R(z)$

- 1: $e \leftarrow P_1(z)$ ▷ Initialize evaluation with first polynomial
- 2: **for** $i = 2$ **to** n **do** ▷ Start from second polynomial
- 3: $e \leftarrow e \cdot P_i(z)$ ▷ Multiply polynomial evaluations into e
- 4: **end for**
- 5: $e \leftarrow e - R(z)$ ▷ Subtract evaluation of R
- 6: **return** e

**4.5 Fiat-Shamir Transformation**

The Fiat-Shamir transformation [FS87] converts an interactive proof into a non-interactive one by replacing the verifier's random challenge with a hash of the protocol transcript.

This protocol, instantiated using the Fiat-Shamir transformation with hash function H modeled as a random oracle, achieves knowledge soundness of $\delta_s + 3(m^2 + 1)2^{-\lambda}$ [BSCS16], where δ_s is the soundness error for the interactive protocol (see Section 4.7.5), and m is the number of hash queries performed (two for our protocol), and λ is the hash output length.

4.6 Non-interactive proof

Let's look at what the protocol looks like after applying Fiat-Shamir transformation.

Algorithm 7 Pairing Verifier: Evaluating remainders**Input:** $P \in \mathbb{G}_1, Q \in \mathbb{G}_2, c \in \mathbb{F}_{q^{12}}, w \in \{0, 1, 2\}, \mathcal{R}$ array of remainders, $Q_{\text{acc}} \in \mathbb{F}_q[x]$ Internal: 27th root of unity, $W, W^2 \in \mathbb{F}_{q^{12}}$, first round challenge, $z \xleftarrow{\$} \mathbb{F}_q$ **Output:** $\text{pairing}(P, Q) \stackrel{?}{=} 1$ Write $s = 6t + 2 = \sum_{i=0}^{L-1} s_i 2^i$ where $s_i \in \{-1, 0, 1\}$ and $s_L = 1$

- 1: $c^{-1} \leftarrow 1/c$ ▷ Inverse Residue witness
 - 2: $f \leftarrow c^{-1}$ ▷ Initialize with inverse residue witness
 - 3: $T \leftarrow Q$ ▷ T_i for each pair for multi-pairing
 - 4: $x \xleftarrow{\$} \mathbb{F}_q$ ▷ Sample evaluation point from field
 - 5: $e_{\text{acc}} \leftarrow 0, a_{\text{rlc}} \leftarrow 1$ ▷ Evaluation accumulator and random linear combination multiplier
- Every operation involving T, P or Q is repeated for T_i, P_i and Q_i for multi-pairing.

Miller loop bits

- 6: **for** $i = L - 2$ to 0 **do**
- 7: $L_{\text{dbl}} \leftarrow l_{T,T}(P), T \leftarrow [2]T$
- 8: $R \leftarrow \mathcal{R}.\text{pop_front}()$ ▷ Fetch R from prover
- 9: **if** $s_i = 0$ **then**
- 10: $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \text{EvalPoly}(R, x, f, f, L_{\text{dbl}})$ ▷ Algo 6, add L_{*i} for each P_i, Q_i
- 11: **else if** $s_i = 1$ **then**
- 12: $L_{\text{add}} \leftarrow l_{T,Q}(P), T \leftarrow T + Q$
- 13: $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \text{EvalPoly}(R, x, f, f, c^{-1}, L_{\text{dbl}}, L_{\text{add}})$ ▷ Algo 6, add L_{*i} for each P_i, Q_i
- 14: **else if** $s_i = -1$ **then**
- 15: $L_{\text{add}} \leftarrow l_{T,-Q}(P), T \leftarrow T - Q$
- 16: $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \text{EvalPoly}(R, x, f, f, c, L_{\text{dbl}}, L_{\text{add}})$ ▷ Algo 6, add L_{*i} for each P_i, Q_i
- 17: **end if**
- 18: $f \leftarrow R, a_{\text{rlc}} \leftarrow a_{\text{rlc}} \cdot z$ ▷ Update random linear combination factor
- 19: **end for**

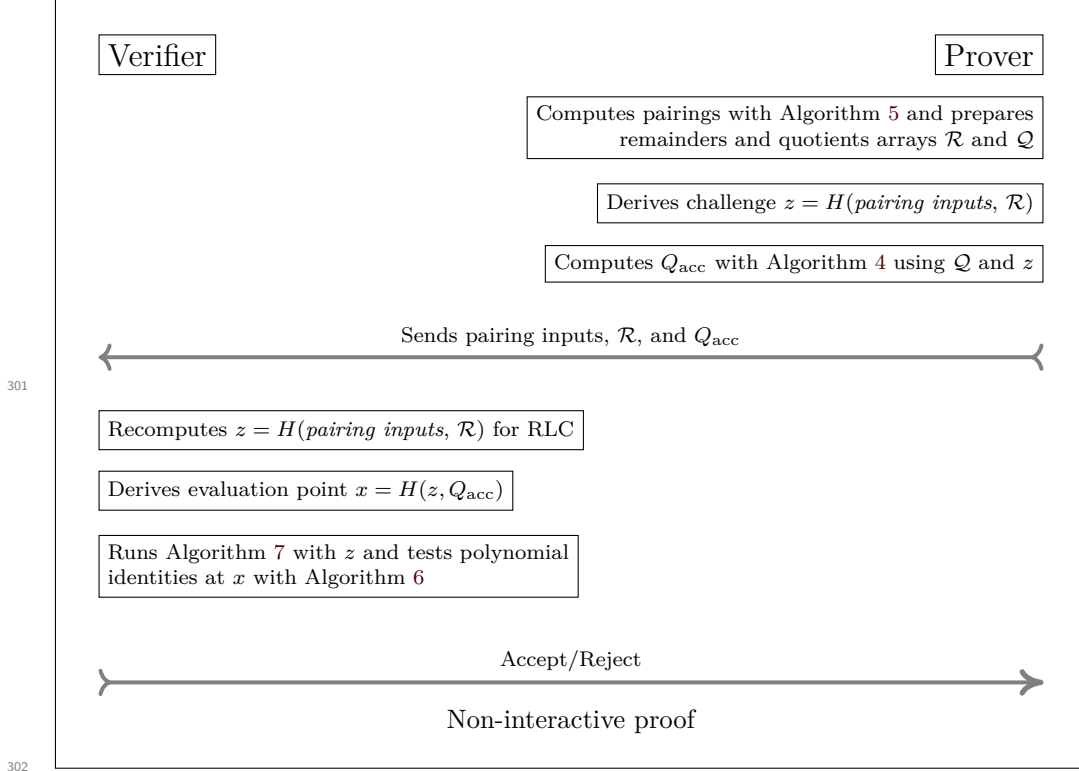
Miller loop Frobenius correction

- 20: $R \leftarrow \mathcal{R}.\text{pop_front}()$ ▷ Fetch R from prover
- 21: $Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$
- 22: $L_{\pi} \leftarrow l_{T,Q_1}(P)$
- 23: $T \leftarrow T + Q_1$
- 24: $L_{-\pi^2} \leftarrow l_{T,-Q_2}(P)$
- 25: $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \text{EvalPoly}(R, x, f, L_{\pi}, L_{-\pi^2})$ ▷ Algo 6, add L_{*i} for each P_i, Q_i
- 26: $f \leftarrow R, a_{\text{rlc}} \leftarrow a_{\text{rlc}} \cdot z$ ▷ Update random linear combination factor

Final exponentiation: Cubic scaling + Frobenius mappings

- 27: $R \leftarrow \mathcal{R}.\text{pop_front}()$ ▷ Fetch R from prover
- 28: **if** $w = 0$ **then**
- 29: $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \text{EvalPoly}(R, x, f, c^{-q}, c^{q^2}, c^{-q^3})$ ▷ Algo 6
- 30: **else if** $w = 1$ **then**
- 31: $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \text{EvalPoly}(R, x, f, W, c^{-q}, c^{q^2}, c^{-q^3})$ ▷ Algo 6
- 32: **else if** $w = 2$ **then**
- 33: $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \text{EvalPoly}(R, x, f, W^2, c^{-q}, c^{q^2}, c^{-q^3})$ ▷ Algo 6
- 34: **end if**

- 35: **return** $Q_{\text{acc}}(x) \stackrel{?}{=} e_{\text{acc}} \wedge R \stackrel{?}{=} 1$ ▷ Polynomial identity and expected pairing output check



Concrete Fiat-Shamir costs: In our Cairo implementation, each POSEIDON hash invocation costs approximately 25-30 Cairo steps. Our batching strategy reduces the number of hash operations from $O(k \cdot n)$ to $O(k)$ where k is the Miller loop length and n is the number of pairings. For BN254 with $k = 66$ and $n = 3$, this yields the 77% reduction observed in Table 7.

4.7 Security Analysis

We analyze the security of our random linear combination batching scheme against a malicious prover attempting to produce an incorrect remainder.

4.7.1 Attack Model

Consider a prover attempting to cheat by submitting an incorrect remainder $R'_j \neq R_j$ within the commitment array $\mathcal{R}$. The prover commits to $\mathcal{R}'$ containing R'_j , after which the challenge z is derived (via Fiat-Shamir) from this commitment or received from the verifier in interactive setting. We have Δ_j as:

$$\Delta_j = R'_j - R_j \neq 0, \text{ and } \deg(\Delta_j) \leq \deg(R_j)$$

4.7.2 Verifier's checks

For the verifier's check to pass with the incorrect remainder, the following equation must hold:

$$\sum_{i=0}^s z^i (A_i - R'_i) \stackrel{?}{=} Q_{\text{acc}} \cdot P_{12}$$

Substituting $R'_j = R_j + \Delta_j$ reveals an imbalance:

$$\sum_{i=0}^s z^i (A_i - R_i) - z^j \Delta_j = Q_{\text{acc}} \cdot P_{12}$$

4.7.3 Absorbing Δ in $\mathcal{R}$

Absorbing Δ_j into the remainders array $\mathcal{R}$ would require the knowledge of z before committing to $\mathcal{R}$. However, since z is derived from the commitment of $\mathcal{R}$ itself (in non-interactive setting) or received after committing to $\mathcal{R}$ (in interactive setting), this temporal ordering prevents the prover from adjusting $\mathcal{R}$ based on z , making it impossible to absorb Δ_j into another R_k in $\mathcal{R}$.

4.7.4 Absorbing Δ in Q_{acc}

Rearranging:

$$\sum_{i=0}^s z^i (A_i - R_i) = Q_{\text{acc}} \cdot P_{12} + z^j \Delta_j$$

For the proof to be accepted, the prover must find a polynomial Q'_{acc} satisfying:

$$\begin{aligned} Q'_{\text{acc}} \cdot P_{12} &= Q_{\text{acc}} \cdot P_{12} + z^j \Delta_j \\ Q'_{\text{acc}} &= Q_{\text{acc}} + \frac{z^j \Delta_j}{P_{12}} \end{aligned}$$

Here P_{12} is irreducible polynomial with $\deg(P_{12}) = 12$, and $\deg(\Delta_j) \leq \deg(R_j) \leq 11$, so we have $\deg(\Delta_j) < \deg(P_{12})$ for all valid j . And since $\deg(\Delta_j) < \deg(P_{12})$, the division $z^j \cdot \Delta_j / P_{12}$ yields a rational function, not a polynomial. When you divide a polynomial of degree lesser than the divisor you get a rational function. So it is **information-theoretically impossible** to find the polynomial Q'_{acc} .

Thus, no polynomial $Q'_{\text{acc}} \in \mathbb{F}_q[x]$ exists that satisfies the verifier's equation with the incorrect remainder R'_j .

4.7.5 Soundness

As seen above, algebraic forgery is impossible. Therefore, security reduces to analysing the probability that a malicious prover succeeds through random chance.

Our random linear combination batching scheme (as described in Section 4.1) effectively converts our univariate polynomial verifications into a single bivariate polynomial identity test:

$$P_{\text{final}}(z, x) = \sum_{i=0}^{n-1} z^i \cdot [\prod_j P_{i,j}(x) - R_i(x)] - Q_{\text{acc}}(x) \cdot P_{12}(x)$$

If a prover commits to incorrect remainder(s), P_{err} becomes a non-zero bivariate polynomial with:

- Degree in z : at most $n - 1$ (from the RLC structure)
- Degree in x : less than 2^8 (from the polynomial products)

By the Schwartz-Zippel lemma (Section 2.2), the probability that P_{err} evaluates to zero at randomly chosen $(z, x) \in \mathbb{F}_q^2$ is bounded by:

$$\delta_s = \Pr[P_{\text{err}}(z, x) = 0] \leq \frac{\deg_z(P_{\text{err}}) + \deg_x(P_{\text{err}})}{|\mathbb{F}_q|} \leq \frac{n + 2^8}{|\mathbb{F}_q|}$$

342 **Concrete parameters for BN254:** With $n = 67$ equations (66 Miller loop steps
343 plus 1 final exponentiation equation) and $|\mathbb{F}_q| \approx 2^{254}$:

$$\delta_s \leq \frac{67 + 256}{2^{254}} < \frac{2^9}{2^{254}} = 2^{-245}$$

344 **Non-interactive proof via Fiat-Shamir:** When transformed into a non-interactive
345 proof with Fiat-Shamir under random oracle model, the knowledge soundness error
346 becomes [BSCS16]:

$$\delta'_s = \delta_s + 3(m^2 + 1) \cdot 2^{-\lambda}$$

347 where $m = 2$ (hash queries) and $\lambda = 254$ (hash output length). Thus:

$$\delta'_s \leq 2^{-245} + 15 \cdot 2^{-254} < 2^{-244}$$

348 4.7.6 Completeness

349 For an honest prover executing Algorithm 5 with valid inputs:

1. **Polynomial construction:** At each operation, the prover computes following polynomials:

$$\prod_j P_{i,j} = Q_i \cdot P_{12} + R_i$$

2. **RLC formation:** Given challenge z , the prover forms:

$$Q_{\text{acc}} = \sum_{i=0}^{n-1} z^i \cdot Q_i$$

3. **Verifier's check:** The verifier evaluates at point x :

$$\begin{aligned} \text{LHS} &= \sum_{i=0}^{n-1} z^i \cdot \left[\prod_j P_{i,j}(x) - R_i(x) \right] \\ &= \sum_{i=0}^{n-1} z^i \cdot Q_i(x) \cdot P_{12}(x) \quad (\text{by construction}) \\ &= P_{12}(x) \cdot \sum_{i=0}^{n-1} z^i \cdot Q_i(x) \\ &= P_{12}(x) \cdot Q_{\text{acc}}(x) = \text{RHS} \end{aligned}$$

355 Since the equality holds for any evaluation point x , the protocol satisfies perfect
356 completeness.

357 5 Conclusion

358 5.1 Target Environments

359 Our polynomial batching approach delivers maximum impact in constraint-based environ-
360 ments where traditional optimization strategies fall short. These environments share key
361 characteristics that make our techniques particularly effective:

362 **Arithmetized Computation Models:** Systems like Cairo VM, R1CS, and Plonkish
363 arithmetization benefit most from our approach. In these environments, field inversions

cost nearly the same as multiplications (2C vs 1C), while polynomial verification through Schwartz-Zippel becomes remarkably efficient. Our batching transforms hundreds of individual field operations into single polynomial verifications.

Calldata-Constrains Blockchains: Ethereum Layer 2s, Bitcoin sidechains, and other blockchain VMs impose limits on transaction calldata. Our 77% reduction in commitment overhead directly translates to fitting more proofs within transaction limits—enabling applications previously impossible due to size constraints.

Recursive Proof Systems: In recursive SNARK/STARK compositions where pairings verify inner proofs, our optimizations compound at each recursion level. The 37% overall speedup enables deeper recursion trees and more complex proof compositions within practical time bounds.

Zero-Knowledge Virtual Machines (ZKVMs): Systems like RISC Zero, SP1, and Jolt that emulate general computation benefit from our unified polynomial framework, as they naturally express operations as constraint systems where our batching excels.

5.2 Implementation and Performance

Table 7 presents measurements from Garaga’s production Cairo implementation on Starknet, demonstrating that our theoretical predictions (from section 3.2) closely match real-world performance.

For 3-pairing Miller loop verification (standard in Groth16), our theoretical analysis (Section 3.2.2) for a single bit operation predicted:

- 47% constraint reduction (76C $\rightarrow$ 40C per operation)
- 75% commitment reduction (48 $\rightarrow$ 12 $\mathbb{F}_p$ elements)

The measured results for Miller Loop validate these predictions:

- **45% reduction** in field multiplications (14,456 $\rightarrow$ 7,975 MULMOD)
- **77% reduction** in hash operations (3,758 $\rightarrow$ 876 POSEIDON)
- **53% reduction** in total execution VM CPU cycles (236,382 $\rightarrow$ 110,666)

The close alignment validates our polynomial batching analysis. The hash reduction exceeds our prediction because non-zero bit operations save more commitments and also the associated Fiat-Shamir challenge derivations.

Calldata and Fiat-Shamir: For Miller loop operations, POSEIDON hash reductions improve from 54% ($n=1$) to 69% ($n=2$) to 77% ($n=3$). This occurs because traditional approaches require $O(k \cdot n)$ commitments where k is the Miller loop length and n is the number of pairings, while our consolidated polynomial approach requires only $O(k)$ commitments regardless of n . Details in Table 7

5.3 Gnark, the current state of the art

For comparison, Gnark, which is considered the current state of the art for field emulation, reports the following constraints for BN254 pairings: single-pair pairing requires 960,784 R1CS constraints, while the Miller loop alone requires 311,672 R1CS constraints. Our optimized implementation achieves 53,130 steps for a single Miller loop and 155,366 steps for a complete pairing, representing a significant improvement in algorithmic efficiency for the verifier.

We intend to implement our protocol over Gnark to enable significant improvements for recursive BN254 SNARKs. However, these two challenges need to be addressed:

Table 7: Performance comparison: Cairo implementation on Starknet

Operation	MULMOD		ADDMOD		POSEIDON		Speedup
	Before	After	Before	After	Before	After	
Miller Loop (BN254)							
Single pair	5,984	3,303	5,927	3,228	1,810	828	47.7%
Three pairs	14,456	7,975	14,463	7,924	3,758	876	53.2%
Average	44.8%		45.2%		76.7%		50.6%
Complete Pairing (BN254)							
Single pair	10,670	7,984	13,150	10,446	3,741	2,759	23.8%
Three pairs	19,142	12,656	21,686	15,142	5,689	2,807	37.1%
Average	33.9%		30.2%		50.7%		30.9%
Miller Loop (BLS12_381)							
Single pair	4,936	2,672	4,966	2,686	1,580	790	47.2%
Three pairs	11,356	6,164	11,608	6,364	3,088	834	53.8%
Average	45.7%		45.5%		73.0%		50.6%
Complete Pairing (BLS12_381)							
Single pair	10,064	7,795	14,027	11,742	3,913	3,123	19.6%
Three pairs	16,484	11,287	20,669	15,420	5,421	3,167	32.7%
Average	31.5%		25.4%		41.6%		26.4%

MULMOD/ADDMOD: Field operations in $\mathbb{F}_p$

POSEIDON: Hash operations for Fiat-Shamir;

Speedup: VM Cycle reduction.

1. In Gnark, hashes can only be batched once per circuit—this poses a challenge for our IOP implementation from Section 5, which requires multiple rounds of Fiat-Shamir hashing.
2. Achieving the same level of hashing improvements requires computing a GKR verification in-circuit. This necessitates further research on the viability of multiple GKR proofs or the exposure of partial hash state finalizations.

5.4 Final Remarks

This work demonstrates that reimagining classical algorithms through the lens of constraint-based computation models can yield substantial improvements. By treating entire computational phases as unified polynomial relations rather than sequences of field operations, we achieved performance gains that make previously impractical applications feasible.

Our techniques, now deployed in production through Garaga on Starknet, have already enabled new privacy-preserving applications on blockchain platforms. As zero-knowledge proofs become increasingly central to blockchain scalability and privacy, optimizations like ours that dramatically reduce verification costs will be essential for widespread adoption.

The success of our polynomial batching approach suggests that many other cryptographic primitives may benefit from similar reconceptualization in constraint-based environments. We hope this work inspires further research into domain-specific optimizations that exploit the unique characteristics of arithmetized computation models.

References

- [BGM⁺10] Jean-Luc Beuchat, Jorge Enrique González-Díaz, Shigeo Mitsunari, Eiji Okamoto, Francisco Rodríguez-Henríquez, and Tadanori Teruya. High-speed

- software implementation of the optimal ate pairing over barreto-naehrig curves. In *Pairing-Based Cryptography – Pairing 2010*, volume 6487 of *Lecture Notes in Computer Science*, pages 21–39. Springer, 2010.
- [BGW⁺22] Dan Boneh, Sergey Gorbunov, Riad S. Wahby, Hoeteck Wee, Christopher A. Wood, and Zhenfei Zhang. BLS Signatures. Internet-draft, Internet Engineering Task Force, June 2022. Work in Progress.
- [BSCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In *Theory of Cryptography – TCC 2016-B*, volume 9986 of *Lecture Notes in Computer Science*, pages 31–60. Springer, 2016.
- [DHM04] Scott Vanstone Darrel Hankerson and Alfred Menezes. *Guide to Elliptic Curve Cryptography*. Springer-Verlag, 2004. Non-adjacent form representation.
- [DL78] Richard A. Demillo and Richard J. Lipton. A probabilistic remark on algebraic program testing. *Information Processing Letters*, 7(4):193–195, 1978.
- [FE] Feltroidprime and Keep Starknet Strange Exploration. Garaga: State-of-the-art elliptic curve tooling and snarks verification for cairo and starknet. <https://github.com/keep-starknet-strange/garaga>.
- [Fel23] Feltroidprime. Faster extension field multiplications for emulated pairing circuits. <https://hackmd.io/@feltroidprime/BleyHHXNT>, 2023. HackMD.
- [FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology – CRYPTO’ 86*, pages 186–194, Berlin, Heidelberg, 1987. Springer Berlin Heidelberg.
- [GGPR13] Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova. Quadratic span programs and succinct nizks without pcps. In *Advances in Cryptology – EUROCRYPT 2013*, pages 626–645. Springer, 2013. Foundational QAP framework for R1CS.
- [GPR21] Lior Goldberg, Shahar Papini, and Michael Riabzev. Cairo – a turing-complete STARK-friendly CPU architecture. Cryptology ePrint Archive, Paper 2021/1063, 2021.
- [Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology – EUROCRYPT 2016*, volume 9666 of *Lecture Notes in Computer Science*, pages 305–326. Springer, 2016.
- [GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. <https://eprint.iacr.org/2019/953>, 2019. Cryptology ePrint Archive, Paper 2019/953.
- [Hou23] Youssef El Housni. Pairings in rank-1 constraint systems. In *Topics in Cryptology – CT-RSA 2023*, volume 13871 of *Lecture Notes in Computer Science*, pages 321–345. Springer, 2023.
- [KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *Advances in Cryptology – ASIACRYPT 2010*, pages 177–194, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.

- 472 [NE24] Andrija Novakovic and Liam Eagen. On proving pairings. <https://eprint.iacr.org/2024/640>, 2024. Cryptology ePrint Archive, Paper 2024/640.
- 473
- 474 [Sch79] Jacob T. Schwartz. Probabilistic algorithms for verification of polynomial
475 identities. In Edward W. Ng, editor, *Symbolic and Algebraic Computation*,
476 pages 200–215, Berlin, Heidelberg, 1979. Springer Berlin Heidelberg.
- 477 [Sch80] J. T. Schwartz. Fast probabilistic algorithms for verification of polynomial
478 identities. *J. ACM*, 27(4):701–717, October 1980.
- 479 [Zip79] Richard Zippel. Probabilistic algorithms for sparse polynomials. In Edward W.
480 Ng, editor, *Symbolic and Algebraic Computation*, pages 216–226, Berlin, Hei-
481 delberg, 1979. Springer Berlin Heidelberg.